



IT & Software Development Department

ເອກະສານຂໍ້ກຳນົດການທົດສອບທັກສະ

ຕຳແໜ່ງ Frontend Engineer (ລະດັບ Senior ຂຶ້ນໄປ)

Technical Assessment Specification (Take-home)

ເລກທີເອກະສານ	___ / ___
ເວີຊັນ	1.0
ວັນທີອອກເອກະສານ	___ / ___ / ____
ເຕັກໂນໂລຢີ	React ຫຼື Next.js, TypeScript
ຜູ້ຮັບຜິດຊອບ	_____
ການຈັດປະເພດ	ເອກະສານພາຍໃນ / ສົ່ງໃຫ້ຜູ້ສະໝັກ

ສາລະລານ

1. ຈຸດປະສົງ ແລະ ຂອບເຂດ.....	3
2. ເງື່ອນໄຂ ແລະ ຂໍ້ກຳນົດທົ່ວໄປ.....	3
3. ໂຈດ A1: ລະບົບຈັດການຂໍ້ມູນພະນັກງານ (Employee CRUD).....	4
4. ໂຈດ A2: ລະບົບສົນທະນາ (Chat) ລະຫວ່າງພະນັກງານ ແລະ Admin.....	4
5. ໂຈດ A3: ຟອມ Dynamic ແບບ 3 ຊັ້ນ (POST API).....	5
6. ສິ່ງທີ່ຕ້ອງສົ່ງ ແລະ ການທົດສອບ.....	5
6.1 ເອກະສານ README.md.....	5
6.2 ການທົດສອບ (Testing).....	5
7. ລາຍການກວດສອບກ່ອນສົ່ງໂຈດ (ສໍາລັບຜູ້ສໍາພາດ).....	6

1. ຈຸດປະສົງ ແລະ ຂອບເຂດ

ເອກະສານສະບັບນີ້ກຳນົດຂໍ້ກຳນົດ, ເງື່ອນໄຂ ແລະ ເກນການປະເມີນ ຂອງການທົດສອບທັກສະແບບ Take-home ສຳລັບຜູ້ສະໝັກຕຳແໜ່ງ Frontend Engineer ລະດັບ Senior ຂຶ້ນໄປ ເພື່ອປະເມີນຄວາມສາມາດໃນການພັດທະນາລະບົບຈົດໂດຍ React ແລະ Next.js, ການເຊື່ອມຕໍ່ API, ການເຮັດວຽກແບບ real-time, ແລະ ການອອກແບບຟອມທີ່ມີຄວາມຊັບຊ້ອນ.

ການທົດສອບປະກອບດ້ວຍ 3 ໂຈດ (A1, A2 ແລະ A3) ທີ່ພັດທະນາຢູ່ໃນ ລະບົບດຽວກັນ (repository ດຽວ). ໂຈດ A2 ແລະ A3 ຕ້ອງຕໍ່ຍອດຈາກໂຈດ A1 ແລະ ບໍ່ຕ້ອງສ້າງແອັບພລິເຄຊັນໃໝ່.

2. ເງື່ອນໄຂ ແລະ ຂໍ້ກຳນົດທົ່ວໄປ

ຫົວຂໍ້	ລາຍລະອຽດ
ໄລຍະເວລາ	
ເຕັກໂນໂລຢີລັກ	React, Next.js (Tanstack Route), TypeScript (strict mode) ສ່ວນທີ່ເຫຼືອສາມາດລົງເພີ່ມໄດ້
API ທີ່ສະໜອງໃຫ້	API_BASE_URL, ເອກະສານ API (Swagger/Postman) ແລະ ບັນຊີທົດສອບ Admin ແລະ Employee ຢ່າງໜ້ອຍ 2 ບັນຊີ ໂດຍຜູ້ສຳພາດເປັນຜູ້ສົ່ງໃຫ້
ວິທີການສົ່ງ	GitHub repository ພ້ອມ README.md
ການທົດສອບໃນ local	ຕ້ອງ run Frontend ໃນເຄື່ອງ local ໄດ້ດ້ວຍຄໍາສັ່ງ npm i && npm dev ແລະ ມີໄຟລ໌ .env.example

ຜູ້ສະໝັກຕ້ອງປະຕິບັດຕາມຂໍ້ກຳນົດຕໍ່ໄປນີ້:

- ບໍ່ຈຳກັດການໃຊ້ library ແຕ່ຕ້ອງອະທິບາຍເຫດຜົນຂອງການເລືອກໃນ README.md.
- ອະນຸຍາດໃຫ້ໃຊ້ເຄື່ອງມື AI ຊ່ວຍໄດ້ ແຕ່ຜູ້ສະໝັກຕ້ອງເຂົ້າໃຈ ແລະ ສາມາດອະທິບາຍໂຄດທຸກສ່ວນໃນຮອບສຳພາດຕໍ່ໄປ.
- ກໍລະນີເວລາບໍ່ພຽງພໍ ໃຫ້ຈັດລຳດັບຄວາມສຳຄັນ ແລະ ລະບຸໃນ README.md ວ່າຕັດສ່ວນໃດອອກ ພ້ອມເຫດຜົນ.
- ກໍລະນີພົບຂໍ້ຜິດພາດ ຫຼື ຂໍ້ມູນບໍ່ຄົບຖ້ວນຂອງ API ໃຫ້ບັນທຶກໄວ້ ພ້ອມວິທີແກ້ໄຂທີ່ເລືອກ.

3. ໂຈດ A1: ລະບົບຈັດການຂໍ້ມູນພະນັກງານ (Employee CRUD)

ໃຫ້ພັດທະນາເວັບແອັບພລິເຄຊັນສໍາລັບຈັດການຂໍ້ມູນພະນັກງານ ໂດຍໃຊ້ API ທີ່ສະໜອງໃຫ້ ແລະ ມີຄຸນສົມບັດດັ່ງນີ້:

- 1) **ການຢືນຢັນຕົວຕົນ ແລະ ສິດ:** ມີໜ້າ Login/Logout; ແບ່ງເປັນ 2 ບົດບາດ ຄື Admin ແລະ Employee. Admin ຈັດການຂໍ້ມູນໄດ້ທັງໝົດ, Employee ແກ້ໄຂໄດ້ສະເພາະໂປຣໄຟລ໌ຂອງຕົນ. ຕ້ອງປ້ອງກັນ route ທາງຝັ່ງ server.
- 2) **ລາຍການພະນັກງານ:** ມີ pagination, sorting, search ແລະ filter ຢ່າງໜ້ອຍ 2 ເງື່ອນໄຂ; ເກັບສະຖານະຂອງ filter/sort/page ໄວ້ໃນ URL.
- 3) **ສ້າງ / ແກ້ໄຂ / ລຶບ:** ຟອມທີ່ມີ validation ແລະ ສະແດງ error ທີ່ມາຈາກ server ຕາມ field; ມີ confirm dialog ກ່ອນລຶບ; ໃຊ້ optimistic update ພ້ອມ rollback; ປ້ອງກັນການ submit ຊ້ຳ.
- 4) **ໜ້າລາຍລະອຽດ:** ໃຊ້ Server Components ແລະ Suspense; ມີ loading, error ແລະ not-found.
- 5) **ຂໍ້ກຳນົດອື່ນ:** ຮອງຮັບ responsive, keyboard ແລະ accessibility ພື້ນຖານ, ບໍ່ໃຊ້ any ໃນ TypeScript, ແລະ ສະແດງຟອນພາສາລາວໄດ້ຖືກຕ້ອງ.

4. ໂຈດ A2: ລະບົບສົນທະນາ (Chat) ລະຫວ່າງພະນັກງານ ແລະ Admin

ໃຫ້ເພີ່ມຄຸນສົມບັດ Chat ເຂົ້າໃນລະບົບຂອງໂຈດ A1 ໂດຍ ໃຊ້ API/Backend ສໍາລັບ Chat ທີ່ຜູ້ສໍາພາດສະໜອງໃຫ້ (REST ແລະ WebSocket/SSE ຕາມເອກະສານ API). ຜູ້ສະໝັກຕ້ອງສາມາດ run frontend ໃນ local ແລ້ວທົດສອບສົນທະນາລະຫວ່າງ Admin ແລະ Employee ໄດ້ຈິງ (ເປີດ 2 browser ຫຼື 2 session).

- 1) **ຝ່າຍ Employee:** ເຫັນ ແລະ ສົນທະນາກັບ Admin ໄດ້ໃນຫ້ອງສົນທະນາຂອງຕົນເອງ.
- 2) **ຝ່າຍ Admin:** ມີ inbox ສະແດງລາຍການສົນທະນາທັງໝົດ ພ້ອມຊື່ ແລະ ຮູບພະນັກງານ (ຈາກ A1), ຂໍ້ຄວາມຫຼ້າສຸດ, ສະແດງວັນເວລາຂອງຂໍ້ຄວາມ, ຈຳນວນທີ່ຍັງບໍ່ໄດ້ອ່ານ ແລະ ຄົ້ນຫາໄດ້.
- 3) **ການສົ່ງຂໍ້ຄວາມ:** ເປັນແບບ real-time; ໃຊ້ optimistic UI ພ້ອມສະຖານະ sending, sent, read ແລະ failed (ມີປຸ່ມລອງໃໝ່); ຂໍ້ຄວາມຕ້ອງບໍ່ຊ້ຳ ແລະ ລຽງລຳດັບຖືກຕ້ອງ; ໂຫຼດປະຫວັດຍ້ອນຫຼັງດ້ວຍ pagination.
- 4) **ການເຊື່ອມຕໍ່:** ຕ້ອງເຊື່ອມຕໍ່ຄືນອັດຕະໂນມັດ (backoff), ສະແດງສະຖານະການເຊື່ອມຕໍ່, ແລະ ຊົ່ງຂໍ້ຄວາມທີ່ຂາດຫາຍຫຼັງເຊື່ອມຕໍ່ຄືນ.
- 5) **ການແຈ້ງເຕືອນ:** ມີ unread badge ໃນຫົວແອັບ ແລະ typing indicator.
- 6) **ຄວາມປອດໄພ:** ປ້ອງກັນ XSS ໃນເນື້ອຫາຂໍ້ຄວາມ; Employee ຕ້ອງບໍ່ສາມາດເຂົ້າເຖິງ chat ຂອງຜູ້ອື່ນ.
- 7) **ຄຸນສົມບັດເສີມ (ບໍ່ບັງຄັບ):** ການແນບໄຟລ໌ ຫຼື ຮູບພາບ, ສະຖານະ online.

5. ໂຈດ A3: ຟອມ Dynamic ແບບ 3 ຊັ້ນ (POST API)

ໃຫ້ພັດທະນາຟອມ dynamic ແບບ nested 3 ຊັ້ນ ແລະ ສົ່ງຂໍ້ມູນທັງໝົດຜ່ານ POST API ເສັ້ນດຽວ. ທີມ Backend ຈະເປັນຜູ້ສ້າງ API ດັ່ງກ່າວ ແລະ ຈະສົ່ງເອກະສານ ພ້ອມໂຄງສ້າງ payload ໃຫ້ຜູ້ສະໜັກພາຍຫຼັງ.

ໂຄງສ້າງຕົວຢ່າງ (ຈະປັບຕາມ API ຈິງ): ໂຄງການ → ໄລຍະ (Phase) → ໜ້າວຽກ (Task)
ໂດຍໃນລະດັບໜ້າວຽກ ໃຫ້ເລືອກພະນັກງານຜູ້ຮັບຜິດຊອບຈາກ API ຂອງ A1.

- 1) **ໂຄງສ້າງ:** ຊັ້ນທີ 1 ເປັນຂໍ້ມູນຫຼັກ; ຊັ້ນທີ 2 ເປັນລາຍການລູກ (ເພີ່ມ, ລຶບ, ລຽງລຳດັບໄດ້); ຊັ້ນທີ 3 ເປັນລາຍການລູກຂອງຊັ້ນທີ 2 (ເພີ່ມ, ລຶບໄດ້).
- 2) **ຄວາມ dynamic:** ບາງ field ຕ້ອງປ່ຽນແປງຕາມຄ່າທີ່ເລືອກ (conditional); ຕົວເລືອກຕ້ອງໂຫຼດຈາກ API (dropdown ທີ່ຄົ້ນຫາໄດ້).
- 3) **Validation:** ຕ້ອງກວດສອບຄົບທຸກຊັ້ນ (ເຊັ່ນ Zod ແລະ React Hook Form) ແລະ ສະແດງ error ຕາມຕຳແໜ່ງທີ່ຖືກຕ້ອງ ລວມທັງ error ທີ່ມາຈາກ server.
- 4) **ການຮັກສາຂໍ້ມູນ:** ຮັກສາ draft ໄວ້ເມື່ອ refresh ໜ້າ ແລະ ແຈ້ງເຕືອນເມື່ອອອກຈາກໜ້າໂດຍຍັງບໍ່ໄດ້ປັນທຶກ.
- 5) **ປະສິດທິພາບ:** ຟອມທີ່ມີຂໍ້ມູນຫຼາຍຮ້ອຍແຖວຕ້ອງບໍ່ຄ້າງ ແລະ ບໍ່ re-render ທັງຟອມເມື່ອພິມ.
- 6) **ການສົ່ງຂໍ້ມູນ:** ປ້ອງກັນການ submit ຊ້ຳ; ມີສະຖານະ loading, success, error; ຮອງຮັບ keyboard ແລະ accessibility.

6. ສິ່ງທີ່ຕ້ອງສົ່ງ ແລະ ການທົດສອບ

6.1 ເອກະສານ README.md

ຕ້ອງປະກອບດ້ວຍ: ວິທີຕິດຕັ້ງ ແລະ run ໂປຣເຈັກ; ສະຖາປັດຕະຍະກຳ; ການຕັດສິນໃຈ ແລະ trade-off (ການຈັດການ state, ການແບ່ງ Server/Client, ເຫດຜົນທີ່ເລືອກ ໃຊ້ເຄື່ອງມືຕ່າງໆ); ບັນຫາທີ່ພົບກັບ API; ແລະ ສິ່ງທີ່ຈະປັບປຸງຖ້າມີເວລາເພີ່ມ.

6.2 ການທົດສອບ (Testing)

- Unit/Integration test ຢ່າງໜ້ອຍ 10 ກໍລະນີ ຄຸນເລື່ອງ: validation, optimistic update ແລະ rollback, ສິດຕາມ role, ການກັນຂໍ້ຄວາມຊ້າ ແລະ ລຳດັບຂໍ້ຄວາມ, ການເຊື່ອມຕໍ່ຄືນ, ແລະ ຟອມ 3 ຊັ້ນ.
- E2E test ຢ່າງໜ້ອຍ 2 flow: (1) CRUD ພະນັກງານ ແລະ (2) ການສືບທອນລະຫວ່າງ Employee ແລະ Admin.
- ປະຫວັດ Git commit ຕ້ອງອ່ານເຂົ້າໃຈໄດ້ ບໍ່ແມ່ນ commit ດຽວທັງໝົດ.

7. ລາຍການກວດສອບກ່ອນສົ່ງໂຈດ

- API ຂອງ A1 ແລະ Chat ພ້ອມໃຊ້ງານ ແລະ ມີບັນຊີ Admin/Employee ຢ່າງໜ້ອຍ 2 ບັນຊີ.
- API ຂອງ A3 (POST ແລະ payload 3 ຊັ້ນ) ພ້ອມໃຊ້ງານ ແລະ ມີເອກະສານ.
- ກຳນົດ Deadline ແລະ ຊ່ອງທາງສອບຖາມ.
- ແຈ້ງນະໂຍບາຍການໃຊ້ AI ແລະ ການເກັບຮັກສາໂຄດຂອງຜູ້ສະໝັກ.